CPU Scheduling ready → run → no special timer mechanism

1. non-preemptive: process 主动 give it up when terminated / waiting I/O / 主动 yeild statement

preemptive: ① Time Mechanism ② waiting/non-waiting give it up ③ priority — user mode / kernel mode

④ complete I/O or some special event

dispatch latency: 开始前 current process 中断 → kernal → next process — 进入OS, 保存信息 context switching

preemptive: ① ② ④

non-preemptive: ① ② ④ 高 → convey effect: ① ② ④

starvation: ② ③ ④

① FCFS ② SJF ③ SRT ④ PR ⑤ RR ⑥ Multi-level (feedback) Queue

CPU utilization; Throughput; Turnaround Time (完成-到达); Waiting Time; Response Time

(大) Multi-Level Queue Scheduling 1个 ready queue ⇒ 多 seperate queues

System; Priority Interactive; RR ⇒ response time; SRT, SJF, 抢/非抢, 看自最优 average wait/turnaround

Batch: FCFS, SRT, SJF ⇒ Turnaround Time

队列划分标准 (2条件): ① Fixed Priority Scheduling ⇒ starvation ② Time slice

① 保证进入执行 ② 保证不会一直执行 high priority

t) Multi-Level Feedback Queue (process 可以在 queue 间 upgrade/downgrade)

Queue 内容 scheduling algorithm, Queue 间也有 (上述①②)

Memory Management

1. CPU 可直接访问内存 (register, cache, MM)

2. Address Binding

(1) Compile time: process 的 address 在 确定有这个 process 时就被固定 VP = MP

(2) Load time: process 的 address 在 disk → MM 后, 被固定 VP = MP → VP → MP 不同 bandstate

(3) Run time: process 的 address 在 执行时, 都可以变化 VP ≠ MP 不是 disk

↳ OS 中 VM 实现的基础

→ 都经历过 Load, 所以 base + offset, process 大小不可超过 Limitation

3. Memory Allocation

low memory: Resident OS; high memory: User process

(1) Contiguous —— Fragmentation 碎片: 用 compaction 解决 MVT 的内部碎片

(2) Non-Contiguous

① Paging: PTBR; 放 base register; TLB: 快查表 ✗ 同时放 查物的 page/frame 编号

② Segment: external fragment

Virtual Memory

Demand Paging ⇒ FIFO, LRU, Optimal Anticipatory Paging ✗ 预取分页

2种 trap { reference is invalid: terminal; page is not in MM: wait }

effective virtual access time = (1−f) × EAT + f × page fault service f = not hit

EAT (effective access time) = t(1 + memory access time) + (1−t)(2 + memory access time) t = hit

Additional reference bit algorithm 每隔固定时间, 从左 right, 换 value 最大的

Second chance algorithm ① 1→0 (二次机会) ② 替换 ③ 遍历找到 1个 bit = 0 的替换, 查下一page ④ 直到找到 1个

Fixed allocation { Equal; Proportional } ⇒ Local page replacement

Variable allocation ⇒ Local & Global page replacement

Thrashing: Process are busy with doing swap page ⇒ CPU utilization ↓

产生的原因: due to 少量 frames ⇒ 大量 page fault; I/O blocking ↑

进程数量 ② global replacement 会放大这种影响 Page fault frequency (PFF)

单 process PF ⇒ 增多 frame

考虑 process 遭受 PF ⇒ 选择一些 suspend

Working-Set: 保证足量的 frame

Δ − 段 time 内, process exchange 页量

size: Δ 内, process 的种类数 > frame size ⇒ 增加 frame

FIFO: Belady's anomaly

Deadlock request → ready queue → scheduling → run → release

Deadlock Characterization

① Mutual exclusion: 互斥, 同一时间, 只有 1个 process 可用 resource

② Hold and Wait: 在满足 other resource 时, 手里不拥有 resource ⇒ resource utilization ↓, starvation

③ no preemptive

④ Circular wait: priority order

---

4 methods to deal with deadlock

① Ostrich approach ② D prevention (预防)

③ D avoidance (避免分配 resource) ⇒ use additional infor

i 事先声明每种 resource 的最大需求数

ii 动态检查资源分配后, system 是否依然处于 safe state

iii 资源分配状态: 由可用资源数, 已分配资源数和 process 最大需求数共同决定

④ D detection: Process Termination: ① 1个切 ② 1个1个切 ⇒ starvation

(检查 + 恢复) Process Rollback: 重生后可能又卡死在上一次死生点

如果 request edge 得到 许可, 就会 reverse direction ⇒ assignment edge

消除有向边判断 deadlock ⇒ 从 Pi 入手, 若其没有 requesting edge 或可以得到满足,

我们便删除 Pi 的 both requesting/assignment edge

坚决在执行前 若存在 cycle, 直接拒绝执行 — 一定无 cycle 才执行

Cooperating System (被) 影响集合内其他 process

Synchronization 保证 cooperation 的有序进行 randomly choose/circular queue

Critical Section Problem

① Mutex Exclusion: 原位: 会发生 collision → 只能互斥性, 不影响有能保证程序跑

② Progress (CS不能空着, 有被有 process 可进) 证明: someone wants to enter, 其中有人能它进

③ Bounded Waiting: 证明: 上请求后, 一定可以进 (有限 time 内 = without forever)

```
int count = 0, int in = 0, int out = 0, semaphore S = 1
```

Producer	Consumer
while (true) do {	while (true) do {
while (count == NUM_BUF) do { };	while (count == 0) do { };
// wait for an unused buffer	// wait for an available item
P(S);	P(S);
buffer[in] = nextProduced;	nextConsumed = buffer[out];
in = (in + 1) % NUM_BUF;	out = (out + 1) % NUM_BUF;
count = count + 1;	count = count - 1;
V(S);	V(S);
}	// consume a stored item
	}

P(S)
Disable Interrupt
S.value = S.value − 1
if (S.value < 0)
block (S.queue)
Enable interrupt

V(S)
Disable Interrupt
S.value = S.value + 1
if (S.value <= 0)
wakeup(S.queue)
Enable interrupt

while (true) do{
while (TestAndSet(lock)) do{};
<CS>
lock = false
<RS>}

boolean TestAndSet (V){
boolean returnvalue = V;
V = true;
return returnvalue}

monitor 长的像: semaphore 结构

外部的 single queue ⇒ P(mutex)

中间只能 1个 process 执行 ⇒ CS

signal { Java: continuous → 主动 exit/wait; standard: wait → 唤醒 }

wait queue ⇒ conditional value

wait ⇒ V(L); P(L) 另一个进程

signal ⇒ V(mutex)

File Concept → disk 上的 smallest unit! 文件是 abstract data type (ADT)

(1) file is a named collection of related information 存储相关信息集合

(2) file infor − 保存在 directory structure 上,

(3) file system: a collection of files with proper storage and directory structure

(4) 1个 disk 可有多个 partition (file system), 但 1个 partition 也可由多个 disk 组成

(5) attribute: Name, Identifier, Type, Location, Size, creator/owner, Protection, Time

(6) file operation: Create, Read, Write, Reposition, Delete, Truncate (清空)

Open: ① 目录结构中找到该文件 ② load from disk to MM; Close: Store from MM to disk

↳ ① file pointer ② file-open count ③ Disk location ④ access right

只有当最后 1个 process 用完关闭 file 才停止计数 — 记录 file 地址, avoid 每次查结构目录开销太大

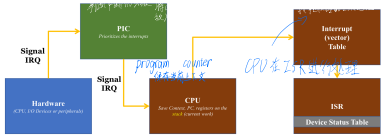
(7) 2 major types of files ① Program files: Source code, Object code, Execution Program

② Data file: Character or ASCII file, Binary file, Free-formatted file, Formatted or structured

(8) { unstructured: 1位串 word/bytes → Unix; structured: Simple record structure (偏定-排-排), complex structure (html) }

Unix 的内核只接收 bytes 字符流, 不关心 file, 但是由程序自己负责 (结构解释)

Magic Number → help with it



CPU 在工作时 不能中断

IRQ – Interrupt Request
PIC – Programmable Interrupt Controller (prioritizes interrupts, and ensures the CPU responds to the most critical ones first)
ISR – Interrupt Service Routine or Interrupt handler
PC – Program Counter

Access Method

Sequential Access: access in order, 支持 read/write next, reset, skip forward/back forward

Direct Access: randomly access), access directly, 支持 Read/Write n 用 Position n + Read/Write next 定位

Index Access: 建立 seperate index file (记 pointers to data block), data is stored in direct/relative file

Directory Structure —— 本质就: file, a collection of nodes containing infor of all files

manager file directory (search, Create, Delete, List, Rename, Traverse 遍历)

1. Single-Level Directory: Name Problem (2文件不可同名), Group Problem (无法文件分类)

2. Two-Level: MFD → UFD, seperate directory for each user, 不同 user 间可有 same name 文件

faster searching; acess rights —— user 只有分别认的文件

3. Tree-Structured: multiple levels, more efficient, grouping capacity 分类分组

current working directory: absolute or relative path names

4. Acyclic-Graph: 允许1个 file/sub-dire 被多个 sub-dire 共享 (aliasing 别名)

Problem: ① double count ② Dangling Pointer —— 删1个 link, file 也没了, 其他 link 变无效

Solution: reference count 创/删 link 时 +1/−1, 直到计数为0, 才删 file

通过性查别名 (aliasing) 修改 file, file 同步更新

5. General-Graph: directory-link − directory 形成 cycle, reference count 失效

① ~~garbage collection~~ ② i 重新保障无环 ii 每目录不允许有 link/path iii 每次添加新 link 都检查环状结构

File Sharing —— ① multi-user system 多用户共同处理同一文件, protection scheme (认证)

② distributed system (via network)

③ remote file system (Manually (FTP, winscp), Automatically (G drive), 共享站 (web))

1. Client Server: 客户端挂载 mount 远程文件系统, 调用系统文件调用接口; 远程执行并返回结果

2. Network-file system (NFS): Unix/Linux 系统下, 最常用的分布式文件系统 — Windows 不支持

3. Storage-Area Network (SAN): 大几容量, 大容量, 高速

4. Cloud-Storage: Over the cloud:

File Storage Allocation

1. Linked Allocation: sequencial access 2. Indexed Allocation: random access

Storage 1. Secondary Storage 2. Disk Storage (local, remote) 3. Terabody 三级

① Host-Attached Storage (most standard arrangement) local

improve fault tolerance —— mirror hard disk (RPID out)

② Network-Attached (NAS): remote procedure call Unix NFS − NAS based

③ Storage-Area Network (SAN): 高 file-sharing, private data network, special storage protocols

Authentication

Knowledge/Ownership/Inherence factor

2 (multi-) factor authentication

certificate, public-key infrastruction

① possession of private key 持私钥 (公钥加密解密)

② 持证机构, 机构签发并颁发证书的证书

Cryptography ⟷ 与 cryptography 互补

1. Transposition cipher 换位重排, 把字母顺序

2. Substitution cipher 用替换表, 字母替换

3. Vigenere cipher 维纳密码

4. Caesar's cipher 每个字母都按固定位移 (A→D, D→B)

Window Scheduling

Window 的 dispatcher 采用 preemptive priority-based RR 调度器 — 32不同的 priorities

real time (16−31), user class (1−15), Memory Management (0)

6个 priority class 高级 REALTIME — IDLE − Priority Class

7个 relative classes TIME-CRITICAL — IDLE

用完成 I/O 的 process 会保留原有的位置 提升 前台进程 (正在用的窗口) 时间片更长 (比后台进程)

普通 process 默认是于 Normal − Priority − Class

Linux Scheduling

140个 priority classes (100 real-time jobs, 40 others)

priority value 动态调整 取决于交互情况: real-time (static), other jobs (dynamic)

时间片配额 → next process → all processes 用完配额 → all 重置

Memory Management

1. Pentium Memory (IA−32) (at most 16384 segments) { 2-level; 1-level } paging 4KB; 4MB

2. ARM Memory VP = MP = 32bit { 2-level; 1-level } paging 4KB; 1MB

3. Itanium Memory VP = 48 bit, MP = 31 bit 4: 4KB 3: 2MB 2: 16GB

Virtual Memory

Window Virtual Memory —— demanding paging with clustering → anticipate surrounding page

发生 page fault: system 取 page 可能会多读其周围的

进程在 MM 中, 不可低于 working set minimum 个 page, 不可高于 xx个 page

若系统低于阈值会回收, 超出 minimum 的部分

单 CPU 采用 delocated clock algorithm, 多 CPU 采用 FIFO

Security Measure level 4个安全防护 level

1. physical level ex: 机房门禁 设备 devices

2. Human level ex: 员工隐私防范

3. OS level ex: file right, process isolation, bug fixed

4. Network level ex: 防止通信被窃听, 防火墙

Computer Security Classification 安全分级

D: Minimal security C: discretionary protection 基于审计的访问保护

支持用户控制自己的操作用户访问 [自主访问控制]

实现相关权限的细粒度访问控制

B: 包含C的全部特性, 每个 object 每次生成强制 label [强制访问控制]

A: 采用形式化设计与验证技术 来确保安全 (最高级)

Information Security CIA

confidentiality / Integrity / Availability

non-repudiation

不可否认性: 你干了什么都会被记录

Protection & Security

1. Protection: mechanisms & policy — OS 执行 code → rule — exi 修改漏洞, 完善防火墙 Internal OS

2. Security 涉及用户认证, 消息完整性, malicious (恶意) 找漏洞的 introduction or flow (漏洞) to external OS

Problem: Intruders/crackers (入侵者) 企图 breach (坏破) security threat (威胁) 潜在安全违规 (Violation) attack: 企图 breach (坏破) security (可以是 accidental or malicious) — 更好防护

Security Violation Categories

1. Bleach of confidentiality: 机密性破坏: 未经授权读取 data (严重危害)

2. Bleach of integrity: 完整性破坏: 未经授权改动 data (严重危害)

3. Bleach of availability: 可用性破坏: 未经授权破坏 data (严重危害)

} Security goal

4. Theft of service: 服务盗用: 未经授权使用 data (严重危害)

5. Denial of service (DOS): 拒绝服务攻击, 阻止合法用户正常使用服务, 会导致垃圾网时攻击 server

---

Breaching Security 4种安全攻击方式

1. Masquerading (伪装冒充): 假充合法用户, 窃取权限和权利

2. Relay attack (重放攻击): 把之前截取的合法信息 重新发送 server, 绕过验证

3. Man-in-the-middle attack (中间人攻击), 在双方 data 加中, 篡改 endor or modify 伪造信息 (公开的中间人)

4. Session hijacking (会话劫持) ex: 劫持已有登录会话, 获取登录权限 (特定的)

Program Threats 5种

1. Trojan horse 木马, 隐藏恶意代码在合法程序中, 伪装善意 程序 "记录鼠标键盘 (黑客) 的密码" 偷取用户 ex: (伪装成) 游戏软件, 但实际 植入 键盘记录

2. Trap door 后门: 开发者/黑客 设置的特殊入口, 可以绕过正常安全检查 快速登录

3. Logic bomb 逻辑炸弹 平时正常运行, 一旦满足某种条件 (如某个日期), 就执行恶意操作

4. Stack & buffer overflow 栈与缓冲区溢出, 利用 program code bug 攻击

Stack 中写入 超出缓冲区容量的 data, 覆盖 stack 上的 函数 返回地址, 调用 return, 跳转到 恶意代码

5. Viruses 病毒 嵌入 合法程序, 或者内核 阶段 具有自我复制能力, 破坏 硬盘 数据 感染 computer

通常针对特定 CPU 架构, OS, app, 常见传播方式: email, macro

Security Measure level 4个安全防护 level

1. physical level ex: 机房门禁 设备 devices

2. Human level ex: 员工隐私防范

3. OS level ex: file right, process isolation, bug fixed

4. Network level ex: 防止通信被窃听, 防火墙

Computer Security Classification 安全分级

D: Minimal security C: discretionary protection 基于审计的访问保护

支持用户控制自己的操作用户访问 [自主访问控制]

实现相关权限的细粒度访问控制

B: 包含C的全部特性, 每个 object 每次生成强制 label [强制访问控制]

A: 采用形式化设计与验证技术 来确保安全 (最高级)

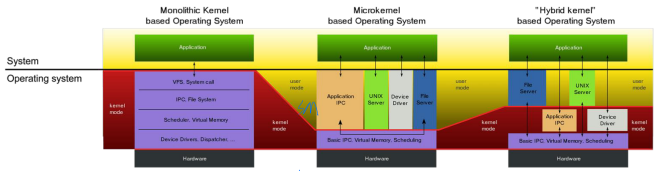
Information Security CIA

confidentiality / Integrity / Availability

non-repudiation

不可否认性: 你干了什么都会被记录

iCloud, dropbox, Google drive.



Linux Virtual Memory —— demand paging → valid/dirty/accessed bit
1个process 1个page table, 不同process可share MM
page finding 机制: kernel swap daemon thread; LRU aging mechanism

Window & *nix Deadlock 检查的overhead远不及deadlock出现频率
Ostrich approach: 直接ignore deadlock

File System → variation of linked allocation
1. DOS File System (FAT): All links are stored in seperate table 在disk开头
2. Window ← 继承
(1) HPFS: 可存入data & special attribute; I-NODE, not first cluster as in FAT
(2) NTFS: journaling/recoverable file system by keep track of transaction ⇒ 快复
Unix hard link, Linux access by open source NTFS server
3. Unix indexed allocation —— Fast file system
file infor 都在 inode中, 不是file structure
4-level index block —— direct/indirect/double indirect/triple indirect block → large file
4. Linux —— Virtual File System 4 main object
① Superblock: global information ② Dentry: 存文件名和Inode的指针
③ Inode: meta file (存目录结构内容), 并指向block ④ File: actual opened file object, an Inode together with a current/writing offset, point to dentry

Synchronization
1. *nix
Message Passing: 普通进程间; Shared Memory: 同1进程中的多线程 Pthread库
Critical Section —— ① 用mutex-locks ② Conditional variables
③ barrier 所有人到齐后, 再一起往下执行
2. Window
Critical Section —— ① 用mutex/semaphore (special critical section) ④ barrier
② Conditional variables ③ 排列 wait function (synchronization objects)
Access Control —— access control list
1. *nix
(1) 单process → 多domain (2) /etc/passwd ✗ ⇒ authentication
(3) all objects in Unix are files (硬件也被抽象成file)
(4) access control rwx —— owner/group/other
2. Window
(1) An item can be a file or a directory and has an owner. con change control list, 让别人用
(2) 相较于*nix的rwx, 区别了 D: delete, P: modify ACL 直接到管理员权限
owner/user 得到权限才可改ACL D: make current account the new owner
(3) Packaging of access privileges into permission sets ⑧ full control
to simplify control: ① no access ② read: RX ③ change: RWXD RWXDPO

Recently Development
(1) Microsoft: open source (2) Win 10: Windows Subsystem full Ubuntu kernel under
(3) Win 11: Powershell → Windows Terminal

1. Input/Output Direction M. keyboard/screen → 改到想要的 (ex: file)
- There is a second output place for programs, called the standard error. It is normally used only for special system error messages and the standard error is also the screen.

> 清空后文件再写入 / 无原文件, 创建新文件再写入
>> 追加 $ GPA < grade2.txt &> transcript.txt
- Input read from grade2.txt and output and error appended to transcript.txt

$ prog1 < infile | prog2 > outfile
- Effect like prog1 < infile > tempfile1; prog2 < tempfile1 > outfile (sequentially)

2. Pipe (concurrently): 在Unix/Linux中, 我们将pipe视为I/O redirection, 但实际不同
Concurrency is about independent computations that can be executed in an arbitrary order with the same outcome. The opposite of concurrent is sequential, meaning that sequential computations depend on being executed step-by-step to produce correct results.

OS operation interrupt在OS中相当重要
1. ① Dormant all OS are interrupt-driven OS受中断驱动 (keyboard)
② interrupt allows OS to get control of system e.x. 强行 强行出死循环
* signal processing for interrupt { no response: terminate; yes: special response }
{ PC <ctrl><Alt><Del> / window <ctrl><c> } In Unix: clock algorithm —— timer handler
2. interrupt security limitation 中断机制的安全分析
① 若 disabling interrupt (禁用中断), OS can't take back CPU
OS不能半路error (死循环), system会 hang-up (ex: 蓝屏) 防止用户使用 interrupt-related操作
P37 ② hardware 的 privileged command, executed by OS & its core component
限制①, 保护 system 安全 security/stability only be executed
4. dual-mode: ① 的具体实现 user 获取OS服务 ⇒ 合法方式
user mode ⇒ normal instruction (no privileged instruction) | kernel mode ⇒ all instruction

System call user can't use privileged instruction
only OS processes could be executing in kernel mode

1. System Program
① OS ② Device Driver ③ Utility Program
④ Compiler ⑤ Assembler
2. Application Program
① Word Processor ② Spreadsheet
③ Web Server ④ Media Player ⑤ Email Client
3. 5 types
① File System: open( ), close( ), read( )
② Process Control: fork( ), exec( )
③ Memory Management/Information maintenance brk( ), mmap( )
④ (Interprocess) Communication: pipe( ), socket( )
⑤ Device Management: set(Console/Mode( )
user execute system call ⇒ trap
CPU switch ⇒ deal with the trap

① File manipulation ② Status infor ③ File Modification ④ Communication
⑤ Programming Language support ⑥ Program loading & executing ⑦ Utilities

OS type & structure
1. ① Batch programming: 批处理 (提前执行) Unix, Linux, Window Server
② Time-sharing: Interleaved manner by CPU, but each has time slice
③ Distributed: multiple-computers, network OS ⇒ { current: 单CPU; parallel: 多CPU; distributed: 多comp }
④ Multi-programming: Interleaved manner by CPU Unix
⑤ Real-time: deadline & penalty
2. *nix { Unix { BSD, System V by AT&T } ; Linux: LAMP development slot-arm PHP Perl Python }
Linux ⇒ fastest supercomputer
Apache
MySQL
3. Advantages of Unix & Linux: ① Reliability ② Security ③ Speed ④ Cost ⑤ Open Source

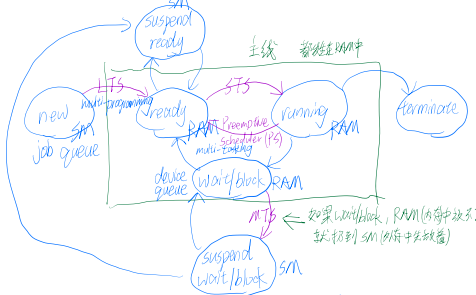
OS structure
1. Unix/Linux (Monolithic Kernel) ① 是一批速度快 ② 封装性强 ③ 体积大
2. Mac OS X (Microkernel) ① slow, IPC传信息时间慢 ② modular 方便增加 ③ 只保留主要部分
Window
Advantages: ① easier to extend a microkernel
② easier to port OS to new architecture/hardware
③ More reliable (less code running in critical kernel mode)
④ More secure
Disadvantages: Performance overhead of user space to kernel space communication
3. Hybird Kernel —— Monolithic 与 Micro 的混合版 (Mac OS)
4. Modular OS —— 本质是 Monolithic, 但有 modular (Linux)
① Object-oriented approach
② Each core component is separated
③ Each talks to the others over known interfaces
④ Each is loadable as needed within the kernel | fail may lead all fail

user — Application Programs — Shell — System Program — System Call — kernel — OS — HW

✓ #!/bin/bash ("$[@:1:4]") ("$[@:5]")
teams=("$@") ⇐ 将input全转成array
✓ declare -A points ⇐ 新array points
for t in "${team[@]}"; do
points["$t"]=0 字符串当下标
done ⇒ 每个team=0 初始化 array points 字典

for file in round*.txt; do
if [! -f "$file"]; then continue; fi;
while IFS read -r line || [-n "$line"]; do 每行遍历
read -r t1 t2 s1 s2 <<< "$line" 划分 匹配team name
if [["${team[$t1]}" = ~ "${t1}" || "${team[$t2]}" = ~ "${t2}"]]; then
if ["$s1" -gt "$s2"]; then
points["$t1"] = $((points["$t1"]+3))
points["$t2"] = $((points["$t2"]+3))
elif [ ]; then
else
fi
fi
done < "$file"
done
for t in "${team[@]}"; do
echo "$t with ${points[$t]} points"
done output

1. 对于 ls -l 以列表形式展开(no hidden)
-a 全部文件名(包括隐藏文件)
-al (全部 + 以列表形式)
隐藏文件 设置文件
① $ ll ← 别名 → 更改空间
comp 的过程 / 目录 | lab 过 可执行文件 | link_to_file 已 符号链接 | no的过程 普通文件
file { symbolic link = 快捷方式; hard link = 副本 }
删除file { symbolic link: 不会在; hard link: 还存在 }
② -o in cc ⇒ to specify the output filename
将源代码文件, 生成可执行的文件名 cc lab1.c -o lab1 ⇒ /lab1
③ alias (别名) 生成 ⇒ alias ls='ls -FC'
2. chmod bb4 filename ⇒ ⊖ rwx r-- r-- → 文件 file
chmod g-r f ⇒ ⊙ rwx r-- r-- → director
group失去read command
user, group, other r=4 w=2 x=1
⇒ 显示结果中, 若 filename 有 x, 那么一定是 x 权限
3. grep 文件搜索 command
grep (1) -i/-h-l (2) "text in file" (3) filename
-i: ignore 大小写
-h: 不显示文件名, 只显示相关内容
-l: 只显示文件名
(2) '^polyu': 以polyu开头
'polyu$': 以polyu结尾
'poly.': . 可表示任何字符
[abc]: only one of a, b, c
[^abc]: 除了a, b, c
(^polyu$): 可多
4. $wc filename ⇒ 4列结果, 展示该file中 line, word, byte, character 数量
5. File Command
$ file filename
$ file filename1 filename2 filename3 ... ⇒ 查看 filenameX 的信息
$ labfiles=lab*.c 存入
$ file $labfiles
$ usefiles="lab*.c *.doc *.pdf" ⇒ 列出信息的名字 —— 相关的文件
$ file $usefiles



① SM → RAM: LTS, 从new中要信息筛选, 放入RAM中
RAM → RAM: STS, 几乎无信息筛选, 选好一个"快"等 (CPU switch)
RAM → SM: MTS, 几乎无信息筛选, 但 RAM → SM 腾出 RAM → RAM 方便
→ throshing (抖动): 大量时间用于内外存的换入, 而用时间在任务上 尽可能多的得 process 放入 RAM
② Type of OS
(1) Multiprogramming: non-preemptive: 等非 process 自己不想干了, process 放入 RAM
(2) Multitasking: preemptive: 其他更高级 process 抢占 他的 CPU
③ Type of Process LTS makes good use of it
{ I/O-bound process: 长时间等待 I/O, 实际经常时间 CPU 使用
CPU-bound process: 长时间占用 CPU 随时间
→ 进程的这 2种 type 并非固定不变, 而是可切换, 动态变化
2. Cooperation: ① information sharing ② Computation speed-up ③ Modularity ④ Convenience
3. Terminated
(1) zombie process (僵尸) 子进程结束, 但父进程未调用 wait() 进行回收 子进程
(2) orphan process (孤儿) 父进程已终止, 但子进程还在继续
→ 使用 Daemon Process (守护进程)
4. exec: path, NULL → execv
execlp: file, NULL → execvp
argv[] → execv: *envp[]

一、Unix/Linux Interprocess Communication
→ Shared Memory Mechanism
1. Communication Method
(1) Direct Communication (like 邮箱)
(2) Indirect Communication (mailbox)
1. IPC Package set up Data Structure in kernel space
① 安全性与隔离
② kernel 统一管理 all process
③ 不相关 process 也可通过 kernel 进行通信
2. Share Memory is not practical in Unix/Linux
① 将 data 放入 address 可能会覆盖原 data, 导致 system crash
Protection Mechanism: OS 当中介, data 不直接访问 memory
data part → OS → memory
② thread within the same process naturally share the memory space
User process use system call (Pthread) create shared memory segment
(2) Message Passing Mechanism
one direction | byte flow 只有byte形式
{ pipe: unidirectional, FIFO, unstructured data stream, kernel 内置 buffer, 点对点, no broadcast
socket: like named pipe (file) }
二、Communication Topology
① star topology (n-1), bottleneck 2 rounds ② Ring topology (n+1) many rounds
③ Linear topology (n+1), many rounds ④ Fully connected $\frac{n(n-1)}{2}$ fast communication
⑤ Tree topology (n-1), more rounds ⑥ Mesh topology $\frac{n(n-1)}{2}$ need routing, flexible
⑦ Hypercube: flexible → $2(k-1) \sim n(n-1)$ ⑧ Bus, not considered with pipes
三、(un)named pipe (本质上是文件file, 但一般用socket替代, not common in Unix/linux Prog)
用mkfifo创建named pipe, 实际是文件file, 不同process可以找到这个file ⇒ heterogenous programs
named pipe is more feasible, while unnamed pipe is more secure
① only parent-child ② no name, cant find in system ③ dead after process dead
unnamed pipe 数量有上限: 因为fd最大1024, fd=0,1,2
Wildcard ⇒ normally used in ls
*: 代替任何1个/1段 字符
?: 代替任何1个字符
[]: 限制一个范围 [1-9] 中的1个字符 [0-2] [0-2A-Z] → [1-2] 无法匹配12 而 {12} 可以
{}: ex: ls lab{1,2,12}.c 精确匹配这三个文件